Disjoint sets: solutions

21-E01 Partitions and repeated UNION calls

Derivation

Starting with six singletons, the partitions after ab, cd, bc, ab, ef, de contain 5,4,3,3,2,1 sets: {ab},{c},{d},{e},{f}; {ab},{cd},{e},{f}; {abcd},{e},{f}; the same after repeated ab; {abcd},{ef}; {abcdef}. There are five successful merges, six UNION calls, and twelve internal finds.

External m=6 MAKE-SET+6 UNION=12; internal finds are not counted again. Expanded into MAKE/FIND/LINK calls, the count is 6+12+5=23, plus constant same-root tests. Repeated same-root LINK can raise the rank of one singleton arbitrarily through UNION(x,x), violating membership at least 2^r and rank$\leq$log n.

Conclusion and checks

The n−1 bound concerns successful merges, not external UNION calls. Equal roots must skip LINK. Preserving the partition alone does not justify arbitrary rank changes.

21-E02 The actual cost of naive lists

Derivation

Naive appending rewrites 1,2,3,4,5 members, totaling 15, with final representative x_6. Weighted union first keeps x_2 on a tie, then appends each new singleton to the existing long list. Each merge rewrites one member, totaling 5, with final representative x_2.

Generally the naive sum is $\Sigma_{i=1}^{n-1} i = n(n-1)/2$, plus n initializations. Thus 2n−1 calls cost $\Theta(n^2)$. The weighted version uses n−1 rewrites and Θ(n) on this input. Equal-size rounds instead move n/2 members per round for $\log_2 n$ rounds, giving Θ(n log n), so the special linear sequence is not universal.

Conclusion and checks

A tail pointer makes next splicing O(1) but does not eliminate member back-pointer updates. Both algorithms produce the same partition, with different representatives and work counts.

21-E03 Per-member growth in an eight-element weighted list

Derivation

For (a,b,c,d,e,f,g,h), rewrite counts are (0,1,1,2,1,2,2,3), totaling 12. After UNION(g,h), UNION(e,g), UNION(a,e), h belongs to sizes 2,4,8 and its set handle follows h→g→e→a.

Disjoint sets: solutions (continued)

For $n=2^k$, round i performs $n/2^i$ merges, each moving 2^{i-1} members, so each round costs n/2 and the total is $(n/2)\log_2 n$. Every moved x belonged to the shorter set, so its new size at least doubles. After j moves, $2^j \le n$, giving $j \le \text{floor}(\log_2 n)$. Charge each element, then sum.

Conclusion and checks

Member h attains three rewrites and a zero. The total for this balanced sequence is 12, not $n \log_2 n = 24$; an upper bound need not be attained by every element.

21-E04 The same operations as a forest

Derivation

Ranks on (a,b,c,d,e,f,g,h) are (3,0,1,0,2,0,1,0), and parents are (a,a,a,c,a,e,e,g). FIND(h) follows h→g→e→a, then redirects h,g,e to a, giving (a,a,a,c,a,e,a,a), without rank changes.

FIND(d) still follows d→c→a and then redirects d. Overwriting parent[h]=a before continuing jumps directly to a and misses g. Node e already points to a, but the incorrect procedure still fails to traverse the original path.

Conclusion and checks

Save old=parent[x] before overwriting. Rank is a historical merge bound, not current depth. Resetting it would invalidate the strict parent-rank invariant used in the proof.

21-E05 Rank is not a lower bound on current height

Derivation

Create four singletons. UNION(a,b) gives rank 1 at a, UNION(c,d) gives rank 1 at c, and UNION(a,c) gives rank 2 at a. Node d still reaches a through c. FIND(d) redirects it, leaving b,c,d directly under a: height1 with rank 2.

Induct on root rank. Rank 0 starts with $1=2^0$ member. Increasing two equal rank-r roots to rank r+1 combines disjoint sets of at least 2^r each, hence at least 2^{r+1}. Unequal-rank merges add members without raising rank. A nonroot retains its old rank, whose historical set required at least 2^r members, so $2^r \le$ final n. This is not a lower bound on its compressed current subtree.

Disjoint sets: solutions (continued)

Conclusion and checks

floor($\log_2 10^6$)=19. Values 0..19 require ceil($\log_2 20$)=5 bits. Parent indices and alignment are additional; five bits do not store the entire node.

21-E06 Numerical levels and iteration counts

Derivation

$A_0(j)=j+1$ gives $A_0^{(i)}(j)=j+i$ and $A_1(j)=2j+1$. Induction gives $A_1^{(i)}(j)=2^i(j+1)-1$, so $A_2(j)=2^{j+1}(j+1)-1$ and $A_3(1)=A_2(A_2(1))=A_2(7)=2047$. Each composition feeds the previous result back as input.

Before compression, g,e,p have ranks 1,2,3 and parent ranks 2,3,4. Each has level0 and iter1. Their potentials are 2,5,8. Node h has rank 0 and potential0; root r has rank 4 and potential12. Path potential totals 27.

After compression, all three parent ranks are 4. For g, $A_1(1)=3\le 4<A_2(1)=7$ gives level1, iter1, phi1. For e, $A_0(2)=3$, $A_0^2(2)=4$, $A_0^3(2)=5>4$ give level0, iter2, phi4. Node p remains level0, iter1, phi8. The endpoint potentials stay fixed; the total becomes 25, a decrease of 2.

Conclusion and checks

Nodes g,e are good because a later nonroot shares their old level. Node p is the last nonroot at that level; h is the rank-zero start and r the root. Raising g's level and e's iter each decreases phi by one. Iter need not be monotone across level changes.

21-E07 Potential accounting needs fixed constants

Derivation

Actual cost is at most 14. Two good nodes give $\Delta\Phi\le -2$, hence $\Delta\Psi\le -6$ and amortized cost≤ 8. Generally $g\ge\max(0,s-(\alpha+2))$, so scaled amortized cost$\le cs-Cg$. For $s\le\alpha+2$ this is at most $c(\alpha+2)$. Otherwise it is $(c-C)s+C(\alpha+2)\le C(\alpha+2)$, proving $O(1+\alpha)$. Fix C throughout the analysis.

The four potential changes are (4,−2,7,−3), giving amortized costs (7,5,9,5), total26. Actual cost totals20. Telescoping gives $\Sigma a=\Sigma c+\Psi_{final}-\Psi_{initial}=20+6-0=26$, hence $\Sigma c=26-6=20\le 26$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Disjoint sets: solutions (continued)

Conclusion and checks

Operations may deposit or spend credit. One nonnegative potential and its total endpoint difference are essential; unmatched big-O and big-Omega coefficients cannot simply cancel.

21-E08 Single calls, sequence cost, and small n

Derivation

For rank alone, $n=2^k$ equal-rank pairwise merges create depth k. Without compression, f repeated finds on that leaf cost $\Theta(f \log n)$, after $O(n)$ construction. Arbitrary LINK can attach each old root below a new singleton, producing length $n-1$ and first-find cost $\Theta(n)$. With both rank and compression, a single find can still cost $\Theta(\log n)$, while m calls total $O(m(1+\alpha(n)))$. Construction and potential pay for the first long find.

Values are $\alpha(0)=0$, $\alpha(2)=0$, $\alpha(3)=1$, $\alpha(7)=2$, $\alpha(8)=3$, $\alpha(2047)=3$, $\alpha(2048)=4$. For n=1 or2, MAKE and FIND still do work, so retain $1+\alpha$. Only $n \geq 3$ permits $O(m\alpha)$.

Conclusion and checks

Single-operation worst-case and sequence-amortized costs use different denominators. The arbitrary linear-chain counterexample omits rank and cannot prove $\Theta(n)$ for a rank-plus-compression find.

21-E09 Minimum-key queries and directed graphs

Derivation

Forcing a smaller-key root can attach a high-rank root under a lower-rank one, breaking strict parent-rank increase and the height proof. Preserve rank linking and store minKey at each root. Initialize it to the singleton key, and set the new root's summary to the minimum of the two old summaries in O(1). Query the minimum by FIND followed by reading root.minKey. The structural root need not itself have the minimum key.

For $a \to b$ alone, direction-blind UNION merges a,b, but b cannot reach a, so the SCCs are {a},{b}. In an undirected graph with only ab, deleting ab after union requires splitting the set. The three operations store neither a split operation nor enough history to restore the old partition. Repeating MAKE(a) violates the existing-element precondition.

Disjoint sets: solutions (continued)

Conclusion and checks

Attach constant-time mergeable summaries to rank roots without changing the linking rule. DSU handles insertion-only undirected connectivity; SCC and edge deletion require other algorithms.

Verification code appendix: disjoint_depth_checks.py

```
"""Linked/forest partition checks and exact capped-Ackermann potential accounting."""
from functools import lru_cache
from random import Random
from math import log2


@lru_cache(None)
def ack(k,j,cap):
    if j>=cap:return cap
    if k==0:return min(cap,j+1)
    value=j
    for _ in range(j+1):
        value=ack(k-1,value,cap)
        if value>=cap:return cap
    return value


def alpha(n):
    k=0
    while ack(k,1,n+1)<n:k+=1
    return k


class Lists:
    def __init__(self,n):
        self.rep=list(range(n));self.members={x:[x] for x in range(n)}
        self.moves=[0]*n;self.sizes=[[] for _ in range(n)]
    def union(self,x,y,weighted=True):
        x=self.rep[x];y=self.rep[y]
        if x==y:return x
        if weighted and len(self.members[x])<len(self.members[y]):x,y=y,x
        old_size=len(self.members[y]);new_size=len(self.members[x])+old_size
        if weighted:assert new_size>=2*old_size
        for z in self.members[y]:
            self.rep[z]=x;self.moves[z]+=1;self.sizes[z].append(new_size)
        self.members[x]+=self.members.pop(y)
        return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disjoint sets: solutions (continued)

```
class Forest:
    def __init__(self,n):
        self.p=list(range(n));self.rank=[0]*n;self.n=n;self.a=alpha(n)
        self.finds=self.good=self.links=0
    def path(self,x):
        out=[x]
        while self.p[x]!=x:x=self.p[x];out.append(x)
        assert len(out)<=self.n
        return out
    def parameters(self,x):
        r=self.rank[x];pr=self.rank[self.p[x]]
        if self.p[x]==x or r==0:return None
        k=0
        while ack(k+1,r,pr+1)<=pr:k+=1
        value=r;i=0
        while True:
            nxt=ack(k,value,pr+1)
            if nxt>pr:break
            value=nxt;i+=1
        assert 0<=k<self.a and 1<=i<=r
        return k,i
    def potential(self):
        total=0
        for x,r in enumerate(self.rank):
            parameter=self.parameters(x)
            value=self.a*r if parameter is None else (self.a-parameter[0])*r-parameter[1]
            assert value>=0;total+=value
        return total
    def find(self,x):
        path=self.path(x);before=self.potential()
        levels={z:self.parameters(z) for z in path[:-1] if self.rank[z]>0}
        good={z for j,z in enumerate(path[:-1]) if z in levels and any(y in levels and
levels[y][0]==levels[z][0] for y in path[j+1:-1])}
        assert len(good)>=max(0,len(path)-(self.a+2))
        root=path[-1]
        for z in path[:-1]:self.p[z]=root
        after=self.potential()
        assert after-before<=-len(good)
        assert 3*len(path)+4*(after-before)<=4*(self.a+2)
        self.finds+=1;self.good+=len(good)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Disjoint sets: solutions (continued)

```
        return root
    def union(self,x,y):
        x=self.find(x);y=self.find(y)
        if x==y:return x
        before=self.potential()
        if self.rank[x]<self.rank[y]:x,y=y,x
        self.p[y]=x
        if self.rank[x]==self.rank[y]:self.rank[x]+=1
        assert self.potential()-before<=self.a
        self.links+=1
        return x
    def check(self):
        groups={}
        for x in range(self.n):
            path=self.path(x);groups.setdefault(path[-1],[]).append(x)
            if self.p[x]!=x:assert self.rank[x]<self.rank[self.p[x]]
            assert len(path)-1<=self.rank[path[-1]]
        for root,members in groups.items():assert len(members)>=2**self.rank[root]
        return groups


def balanced(structure,n):
    size=1
    while size<n:
        for x in range(0,n,2*size):structure.union(x,x+size)
        size*=2


def main():
    rng=Random(21024);operations=0;good=0
    for n in [1,2,3,8,17,64]:
        f=Forest(n);linked=Lists(n);labels=list(range(n))
        for _ in range(600):
            x=rng.randrange(n);y=rng.randrange(n)
            if rng.random()<.55:
                old=labels[y];new=labels[x]
                labels=[new if z==old else z for z in labels]
                f.union(x,y);linked.union(x,y)
            else:f.find(x)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disjoint sets: solutions (continued)

```
            roots=[f.path(i)[-1] for i in range(n)]
            assert all((roots[i]==roots[j])==(labels[i]==labels[j])==
(linked.rep[i]==linked.rep[j]) for i in range(n) for j in range(n))
            f.check();operations+=1
        assert max(linked.moves,default=0)<=int(log2(n))
        good+=f.good
    for n in [2,4,8,16,32,128]:
        l=Lists(n);balanced(l,n)
        assert sum(l.moves)==n*int(log2(n))//2
        assert max(l.moves)==int(log2(n))
        f=Forest(n);balanced(f,n);before=f.rank[:]
        assert f.path(n-1)==[n-1]+[n-2**i for i in range(1,int(log2(n))+1)]
        for x in range(n):f.find(x)
        assert f.rank==before and all(len(f.path(x))<=2 for x in range(n))
        f.check();good+=f.good
    for n in range(1,33):
        naive=Lists(n);weighted=Lists(n)
        for x in range(1,n):naive.union(x,x-1,False);weighted.union(x,x-1)
        assert sum(naive.moves)==n*(n-1)//2
        assert sum(weighted.moves)==max(0,n-1)
    f=Forest(8);balanced(f,8)
    assert f.path(7)==[7,6,4,0] and f.rank==[3,0,1,0,2,0,1,0]
    f.find(7);assert [f.p[z] for z in [7,6,4]]==[0,0,0]
    assert [alpha(n) for n in [0,1,2,3,4,7,8,2047,2048]]==[0,0,0,1,2,2,3,3,4]
    assert ack(1,3,100000)==7 and ack(2,3,100000)==63
    f=Forest(16);balanced(f,16);old=f.potential()
    assert f.path(15)==[15,14,12,8,0]
    assert [f.parameters(x) for x in [14,12,8]]==[(0,1),(0,1),(0,1)]
    f.find(15)
    assert [f.parameters(x) for x in [14,12,8]]==[(1,1),(0,2),(0,1)]
    assert old-f.potential()==2
    print(f'PASS mixed_partition_operations={operations}
list_and_forest_vs_explicit_equivalence=True '
          f'exact_potential_good_nodes={good} FIND_and_LINK_charge_inequalities=True '
          'balanced_moves_n8=12 naive_moves_n32=496 weighted_moves_n32=31 '
          'sample_h_g_e_a_ranks=0,1,2,3 compression_ranks_unchanged=True
alpha_small_bases=True')


if __name__=='__main__':main()
```

Disjoint sets: solutions (continued)

Verification code appendix: union_rank_path_compression.py

```
"""Check the chosen inverse-Ackermann definition and path compression."""

from dataclasses import dataclass


def iterate(function, value, times):
    for _ in range(times):
        value = function(value)
    return value


def ackermann_level(j, level):
    if level == 0:
        return j + 1
    if level == 1:
        return 2 * j + 1
    if level == 2:
        return (1 << (j + 1)) * (j + 1) - 1

    def previous(value):
        return ackermann_level(value, level - 1)

    return iterate(previous, j, j + 1)


def alpha_through_three(n):
    level = 0
    while level <= 3 and ackermann_level(1, level) < n:
        level += 1
    assert level <= 3
    return level


@dataclass
class Node:
    name: str
    parent: object = None
    rank: int = 0

    def __post_init__(self):
        if self.parent is None:
            self.parent = self


def find(node):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Disjoint sets: solutions (continued)

```
    path = []
    while node.parent is not node:
        path.append(node)
        node = node.parent
    for item in path:
        item.parent = node
    return node


def union(a, b):
    root_a, root_b = find(a), find(b)
    if root_a is root_b:
        return root_a
    if root_a.rank > root_b.rank:
        root_a, root_b = root_b, root_a
    root_a.parent = root_b
    if root_a.rank == root_b.rank:
        root_b.rank += 1
    return root_b


def run():
    expected = {2: 0, 3: 1, 7: 2, 8: 3, 2047: 3}
    for n, level in expected.items():
        assert alpha_through_three(n) == level, (n, alpha_through_three(n), level)
    assert ackermann_level(1, 3) == 2047
    assert ackermann_level(1, 3) < 2048
    # A4(1)=A3(A3(1)); A3 is strictly increasing and A3(1)=2047,
    # so the next value is at least 2048. The huge value is not materialized.
    assert 2048 == ackermann_level(1, 3) + 1

    root, p, x = Node("root", rank=2), Node("p", rank=1), Node("x", rank=0)
    p.parent = root
    x.parent = p
    assert find(x) is root
    assert x.parent is root and p.parent is root
    assert (x.rank, p.rank, root.rank) == (0, 1, 2)
    print("PASS ackermann cases=5 alpha2048=4_proved path_compressed=2 ranks_unchanged=3")
    return expected


if __name__ == "__main__":
    run()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study